

Introduction to Transaction Processing Concepts and Theory

Introduction to Transaction Processing

- **Transaction:** An executing program (process) that includes one or more database access operations
 - Read operations (database retrieval, such as SQL SELECT)
 - Write operations (modify database, such as SQL INSERT, UPDATE, DELETE)
 - Transaction: A logical unit of database processing
 - Example: Bank balance transfer of \$100 dollars from a checking account to a saving account in a BANK database
- **Note:** Each execution of a program is a *distinct transaction* with different parameters
 - Bank transfer program parameters: savings account number, checking account number, transfer amount

Introduction to Transaction Processing (cont.)

- A transaction (set of operations) may be:
 - stand-alone, specified in a high level language like SQL submitted interactively, or
 - consist of database operations embedded within a program (most transactions)
- **Transaction boundaries:** Begin and End transaction.
 - Note: An **application program** may contain several transactions separated by Begin and End transaction boundaries

Introduction to Transaction Processing (cont.)

- **Transaction Processing Systems:** Large multi-user database systems supporting thousands of *concurrent transactions* (user processes) per minute
- **Two Modes of Concurrency**
 - **Interleaved processing:** concurrent execution of processes is interleaved in a single CPU
 - **Parallel processing:** processes are concurrently executed in multiple CPUs (Figure 21.1)
 - Basic transaction processing theory assumes interleaved concurrency

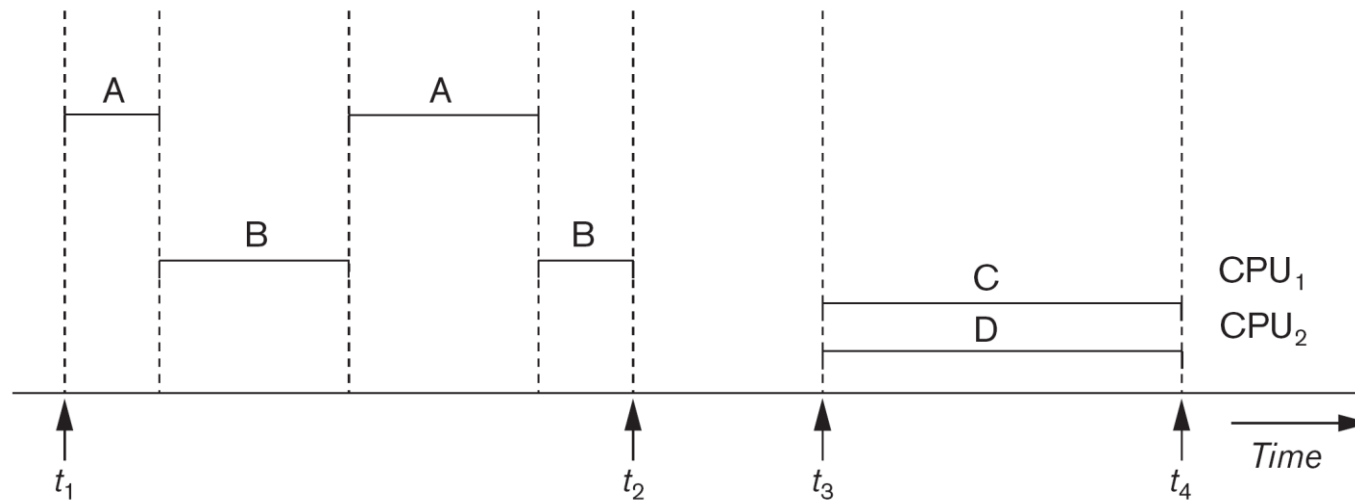


Figure 21.1
Interleaved processing versus parallel processing of concurrent transactions.

Introduction to Transaction Processing (cont.)

READ AND WRITE OPERATIONS:

- Basic unit of data transfer from the disk to the computer main memory is one disk block (or page). A data item X (what is read or written) will usually be the field of some record in the database, although it may be a larger unit such as a whole record or even a whole block.
- **read_item(X) command includes the following steps:**
 - Find the address of the disk block that contains item X.
 - Copy that disk block into a buffer in main memory (if that disk block is not already in some main memory buffer).
 - Copy item X from the buffer to the program variable named X.

Introduction to Transaction Processing (cont.)

READ AND WRITE OPERATIONS (cont.):

- **write_item(X) command includes the following steps:**
 - Find the address of the disk block that contains item X.
 - Copy that disk block into a buffer in main memory (if it is not already in some main memory buffer).
 - Copy item X from the program variable named X into its correct location in the buffer.
 - Store the updated block from the buffer back to disk (either immediately or at some later point in time).

Transaction Notation

- Figure below shows two examples of transactions
- Notation focuses on the read and write operations
- Can also write in shorthand notation:
 - T1: b1; r1(X); w1(X); r1(Y); w1(Y); e1;
 - T2: b2; r2(X); w2(X); e2;
- bi and ei specify transaction boundaries (begin and end)
- i specifies a unique transaction identifier (Tid)

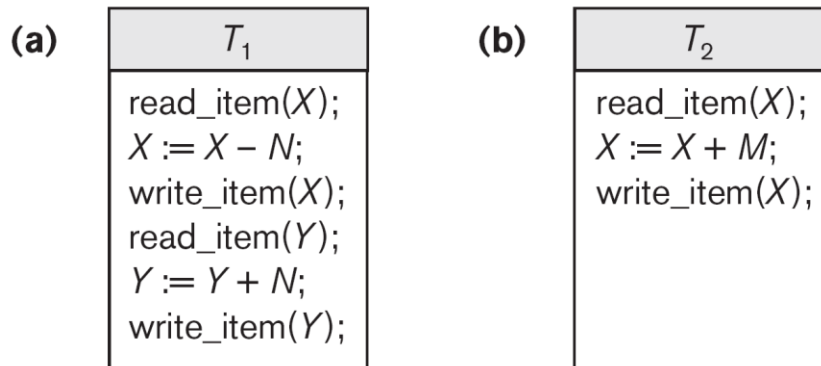


Figure 21.2

Two sample transactions. (a) Transaction T_1 . (b) Transaction T_2 .

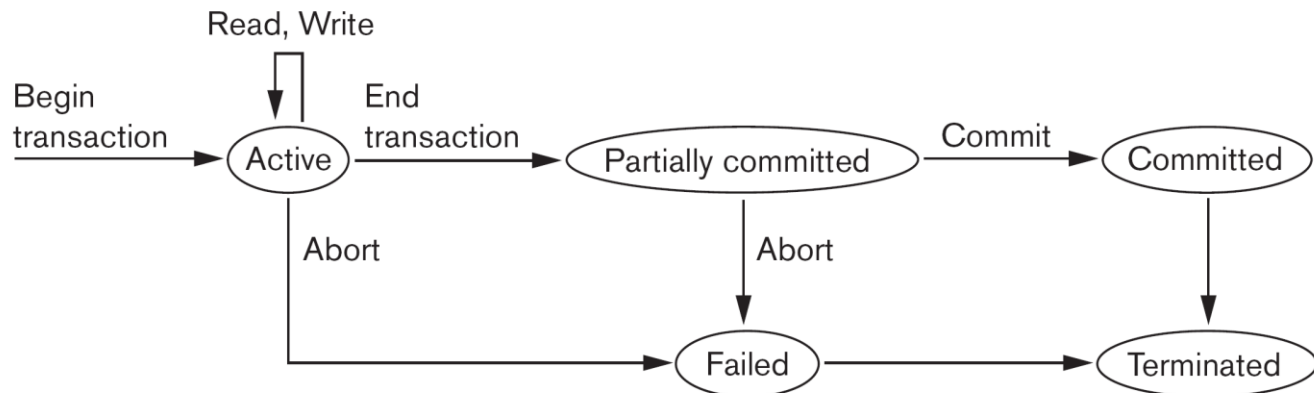
Transaction and System Concepts

A **transaction** is an atomic unit of work that is either completed in its entirety or not done at all. A transaction passes through several states.

Transaction states:

- Active state (executing read, write operations)
- Partially committed state (ended but waiting for system checks to determine success or failure)
- Committed state (transaction succeeded)
- Failed state (transaction failed, must be rolled back)
- Terminated State (transaction leaves system)

State transition diagram illustrating the states for transaction execution.



Desirable Properties of Transactions

Called ACID properties – Atomicity, Consistency, Isolation, Durability:

- **Atomicity:** A transaction is an atomic unit of processing; it is either performed in its entirety or not performed at all. Enforced by the recovery protocol.
- **Consistency preservation:** A correct execution of the transaction must take the database from one consistent state to another.
- Specifies that each transaction does a correct action on the database *on its own*. Application programmers and DBMS constraint enforcement are responsible for this.

Desirable Properties of Transactions (cont.)

ACID properties (cont.):

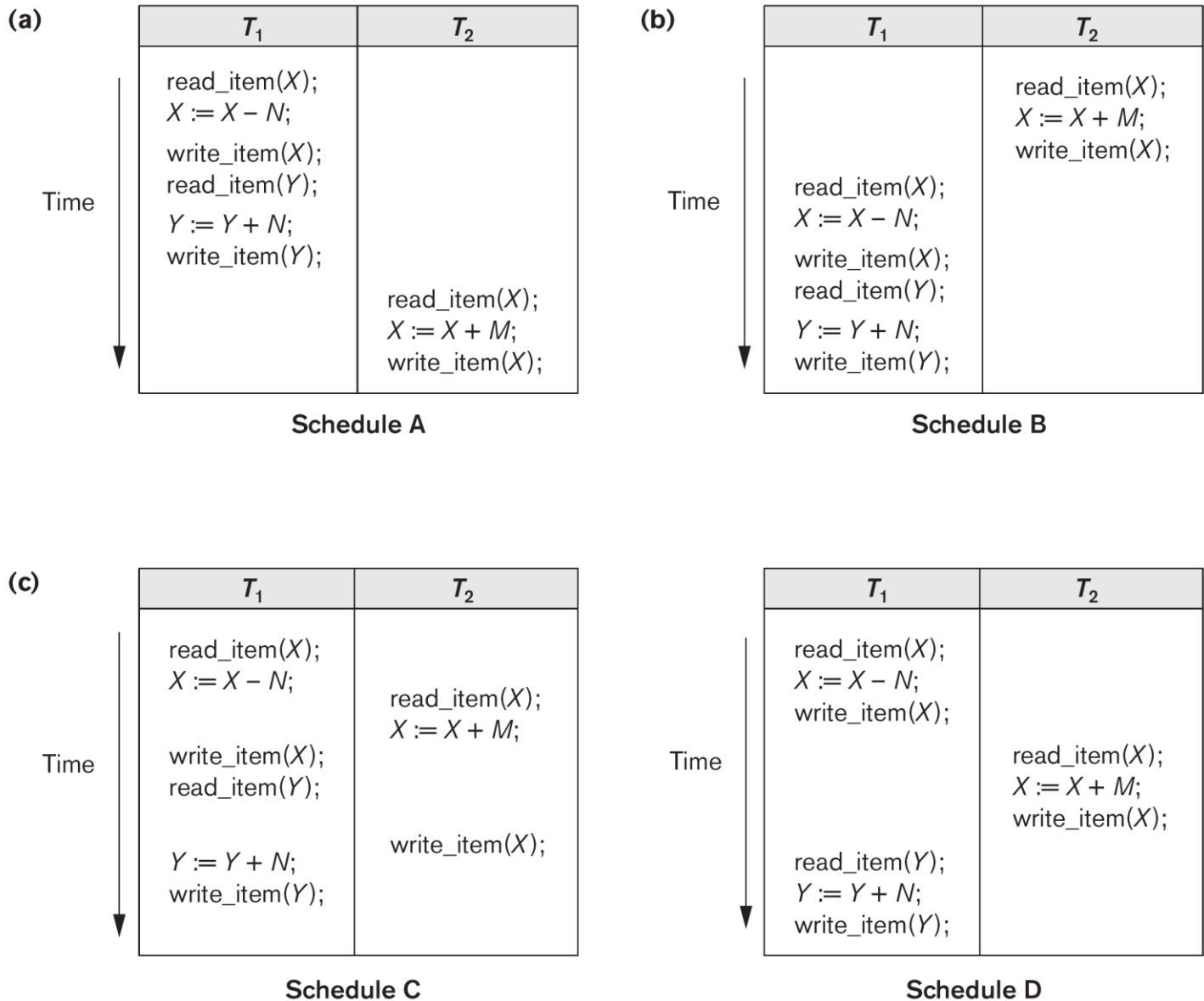
- **Isolation:** Even though transactions are executing concurrently, they should appear to be executed in isolation – that is, their final effect should be as if each transaction was executed in isolation from start to finish.
- Responsibility of the concurrency control protocol.
- **Durability or permanency:** Once a transaction is committed, its changes (writes) applied to the database must never be lost because of subsequent failure. Enforced by the recovery protocol.

Schedules of Transactions

- **Transaction schedule (or history):** When transactions are executing concurrently in an interleaved fashion, the *order of execution* of operations from the various transactions forms what is known as a **transaction schedule** (or history).
- Figure 21.5 (next slide) shows 4 possible schedules (A, B, C, D) of two transactions T1 and T2:
 - Order of operations from top to bottom
 - Each schedule includes *same operations*
 - Different *order of operations* in each schedule

Figure 21.5

Examples of serial and nonserial schedules involving transactions T_1 and T_2 . (a) Serial schedule A: T_1 followed by T_2 . (b) Serial schedule B: T_2 followed by T_1 . (c) Two nonserial schedules C and D with interleaving of operations.



Schedules of Transactions (cont.)

- Schedules can also be displayed in more compact notation
- Order of operations from left to right
- Include only read (r) and write (w) operations, with transaction id (1, 2, ...) and item name (X, Y, ...)
- Can also include other operations such as b (begin), e (end), c (commit), a (abort)
- Schedules in Figure 21.5 would be displayed as follows:
 - Schedule A: r1(X); w1(X); r1(Y); w1(Y); r2(X); w2(x);
 - Schedule B: r2(X); w2(X); r1(X); w1(X); r1(Y); w1(Y);
 - Schedule C: r1(X); r2(X); w1(X); r1(Y); w2(X); w1(Y);
 - Schedule D: r1(X); w1(X); r2(X); w2(X); r1(Y); w1(Y);

Schedules of Transactions (cont.)

- Formal definition of a **schedule** (or **history**) S of n transactions $T_1, T_2, \dots, T_n$:
An ordering of all the operations of the transactions subject to the constraint that, for each transaction T_i that participates in S , the operations of T_i in S must appear *in the same order* in which they occur in T_i .

Note: Operations from other transactions T_j can be interleaved with the operations of T_i in S .

Schedules of Transactions (cont.)

- For n transactions $T_1, T_2, \dots, T_n$, where each T_i has m_i read and write operations, the number of possible schedules is (! is *factorial* function):
$$(m_1 + m_2 + \dots + m_n)! / ((m_1)! * (m_2)! * \dots * (m_n)!)$$
- Generally very large number of possible schedules
- Some schedules are easy to recover from after a failure, while others are not
- Some schedules produce correct results, while others produce incorrect results
- Rest of chapter characterizes schedules by classifying them based on ease of recovery (**recoverability**) and correctness (**serializability**)

Characterizing Schedules based on Recoverability

Schedules classified into two main classes:

- **Recoverable schedule:** One where no *committed* transaction needs to be rolled back (aborted).
A schedule S is **recoverable** if no transaction T in S commits until all transactions T' that have written an item that T reads have committed.
- **Non-recoverable schedule:** A schedule where a committed transaction may have to be rolled back during recovery.
This violates **Durability** from ACID properties (a committed transaction cannot be rolled back) and so non-recoverable schedules *should not be allowed*.

Characterizing Schedules Based on Recoverability (cont.)

- **Example:** Schedule A below is **non-recoverable** because T2 reads the value of X that was written by T1, but then T2 commits before T1 commits or aborts
- To make it **recoverable**, the commit of T2 (c2) must be delayed until T1 either commits, or aborts (Schedule B)
- If T1 commits, T2 can commit
- If T1 aborts, T2 must also abort because it read a value that was written by T1; this value must be undone (reset to its old value) when T1 is aborted
 - known as *cascading rollback*
- Schedule A: r1(X); w1(X); r2(X); w2(X); c2; r1(Y); w1(Y); c1 (or a1)
- Schedule B: r1(X); w1(X); r2(X); w2(X); r1(Y); w1(Y); c1 (or a1); ...

Characterizing Schedules based on Recoverability (cont.)

Recoverable schedules can be further refined:

- **Cascadeless schedule:** A schedule in which a transaction T2 cannot read an item X until the transaction T1 that last wrote X has committed.
- The set of cascadeless schedules is a *subset of* the set of recoverable schedules.

Schedules requiring cascaded rollback: A schedule in which an uncommitted transaction T2 that read an item that was written by a failed transaction T1 must be rolled back.

Characterizing Schedules Based on Recoverability (cont.)

- **Example:** Schedule B below is **not cascadeless** because T2 reads the value of X that was written by T1 before T1 commits
- If T1 aborts (fails), T2 must also be aborted (rolled back) resulting in *cascading rollback*
- To make it **cascadeless**, the r2(X) of T2 must be delayed until T1 commits (or aborts and rolls back the value of X to its previous value) – see Schedule C
- Schedule B: r1(X); w1(X); r2(X); w2(X); r1(Y); w1(Y); c1 (or a1);
- Schedule C: r1(X); w1(X); r1(Y); w1(Y); c1; r2(X); w2(X); ...

Characterizing Schedules based on Recoverability (cont.)

Cascadeless schedules can be further refined:

- **Strict schedule:** A schedule in which a transaction T_2 can neither read *nor write* an item X until the transaction T_1 that last wrote X has committed.
- The set of strict schedules is a *subset of* the set of cascadeless schedules.
- If *blind writes* are not allowed, all cascadeless schedules are also strict

Blind write: A write operation $w_2(X)$ that is not preceded by a read $r_2(X)$.

Characterizing Schedules Based on Recoverability (cont.)

- **Example:** Schedule C below is **cascadeless** and also **strict** (because it has no blind writes)
- Schedule D is cascadeless, but not strict (because of the blind write $w3(X)$, which writes the value of X before $T1$ commits)
- To make it strict, $w3(X)$ must be delayed until after $T1$ commits – see Schedule E
- Schedule C: $r1(X); w1(X); r1(Y); w1(Y); c1; r2(X); w2(X); \dots$
- Schedule D: $r1(X); w1(X); \underline{w3(X)}; r1(Y); w1(Y); c1; r2(X); w2(X); \dots$
- Schedule E: $r1(X); w1(X); r1(Y); w1(Y); c1; \underline{w3(X)}; r2(X); w2(X); \dots$

Characterizing Schedules based on Serializability

- Among the large set of possible schedules, we want to characterize which schedules are *guaranteed to give a correct result*
- The **consistency preservation** property of the ACID properties states that: each transaction if executed on its own (from start to finish) will transform a consistent state of the database into another consistent state
- Hence, each transaction is *correct* on its own

Characterizing Schedules based on Serializability (cont.)

- **Serial schedule:** A schedule S is **serial** if, for every transaction T participating in the schedule, all the operations of T are executed consecutively (without interleaving of operations from other transactions) in the schedule. Otherwise, the schedule is called **nonserial**.
- Based on the consistency preservation property, *any serial schedule will produce a correct result* (assuming no inter-dependencies among different transactions)

Characterizing Schedules based on Serializability (cont.)

- Serial schedules are *not feasible* for performance reasons:
 - No interleaving of operations
 - Long transactions force other transactions to wait
 - System cannot switch to other transaction when a transaction is waiting for disk I/O or any other event
 - Need to allow concurrency with interleaving without sacrificing correctness

Characterizing Schedules based on Serializability (cont.)

- **Serializable schedule:** A schedule S is **serializable** if it is **equivalent** to some serial schedule of the same n transactions.
- There are $(n)!$ serial schedules for n transactions – a serializable schedule can be equivalent to *any of the serial schedules*
- **Question:** How do we define equivalence of schedules?

Types of Serializability

Serializability of any non-serial schedule can be verified using two types:

- ❑ **Conflict Serializability**
- ❑ **View Serializability.**

Conflict Serializability

- **Conflict Serializability:** A non-serial schedule is a conflict serializable if, after performing some swapping on the non-conflicting operation results in a serial schedule. It is checked using the non-serial schedule and an equivalent serial schedule. This process of checking is called **Conflict Serializability** in DBMS .
- It is tedious to use if we have many operations and transactions as it requires a lot of swapping.
- For checking, we will use the Precedence Graph technique (discussed later in slide number 36).

Equivalence of Schedules (cont.)

- **Conflict equivalent:** Two schedules are conflict equivalent if the relative order of *any two conflicting operations* is the same in both schedules.
- Commonly used definition of schedule equivalence
- Two operations are **conflicting** if:
 - They access the same data item X
 - They are from two different transactions
 - At least one is a write operation
- Read-Write conflict example: $r_1(X)$ and $w_2(X)$
- Write-write conflict example: $w_1(Y)$ and $w_2(Y)$
- Write-Read conflict example: $w_1(X)$ and $r_2(X)$

Equivalence of Schedules (cont.)

- Changing the order of conflicting operations generally *causes a different outcome*
- **Example:** changing $r1(X); w2(X)$ to $w2(X); r1(X)$ means that T1 will read *a different value for X*
- **Example:** changing $w1(Y); w2(Y)$ to $w2(Y); w1(Y)$ means that the final value for Y in the database can be different
- Note that read operations are **not conflicting**; changing $r1(Z); r2(Z)$ to $r2(Z); r1(Z)$ does not change the outcome

Characterizing Schedules Based on Serializability (cont.)

- **Conflict equivalence** of schedules is used to determine which schedules are correct in general (serializable)

A schedule S is said to be **serializable** if it is conflict equivalent to some serial schedule S' .

Characterizing Schedules based on Serializability (cont.)

- A serializable schedule is considered to be correct because it is equivalent to a serial schedule, and any serial schedule is considered to be correct
 - It will leave the database in a consistent state.
 - The interleaving is appropriate and will result in a state as if the transactions were serially executed, yet will achieve efficiency due to concurrent execution and interleaving of operations from different transactions.

Benefits of Serializability in DBMS

Now that we understand the serializability in DBMS let's look at some benefits of Serializability in DBMS.

1. **Predictable Executions:** Since all threads are executed one at a time, there are no surprises. All variables are updated as expected, and no data is lost or corrupted.
2. **Easier to Reason about & Debug:** As each thread is executed alone, it is easier to reason about what each thread is doing and why. This can make debugging much easier since you don't have to worry about concurrency issues.
3. **Reduced Costs:** Serialization in DBMS can help reduce hardware costs by allowing fewer resources to be used for a given computation (e.g., only one CPU instead of two). Additionally, it can help reduce software development costs by making it easier to reason about code and reducing the need for extensive testing with multiple threads running concurrently.
4. **Increased Performance:** In some cases, serializable executions can perform better than their non-serializable counterparts since they allow the developer to optimize their code for performance.
5. **Maintain Consistency in the database:** Due to serializability, non-serial schedule maintain a consistency in the database.

Conflict Equivalent

Non-serial schedule

T1	T2
Read(A) Write(A)	Read(A) Write(A)
Read(B) Write(B)	

Schedule S1

Serial Schedule

T1	T2
Read(A) Write(A) Read(B) Write(B)	Read(A) Write(A) Read(B) Write(B)

Schedule S2

Conflict Equivalent Con't..

After swapping of non-conflict operations, the schedule S1 becomes conflict equivalent as shown below:

T1	T2
Read(A) Write(A) Read(B) Write(B)	Read(A) Write(A) Read(B) Write(B)

Testing for Conflict Serializability

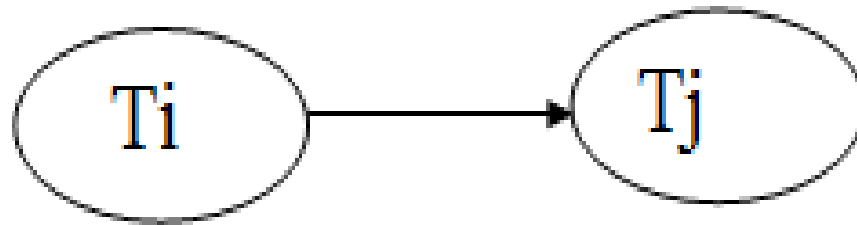
- To test the serializability of a schedule, we can use **Serialization Graph** or **Precedence Graph**. A serialization Graph is nothing but a Directed Graph of the entire transactions of a schedule.
- It can be defined as a Graph $G(V, E)$ consisting of a set of directed-edges $E = \{E1, E2, E3, \dots, En\}$ and a set of vertices $V = \{V1, V2, V3, \dots, Vn\}$. The set of edges contains one of the two operations - READ, WRITE performed by a certain transaction.

Testing for Conflict Serializability

- Serialization Graph is used to test the Serializability of a schedule.
- Assume a schedule S . For S , we construct a graph known as precedence graph. This graph has a pair $G = (V, E)$, where V consists a set of vertices, and E consists a set of edges. The set of vertices is used to contain all the transactions participating in the schedule. The set of edges is used to contain all edges $T_i \rightarrow T_j$ for which one of the three conditions holds:
 1. Create a node $T_i \rightarrow T_j$ if T_i executes write (Q) before T_j executes read (Q).
 2. Create a node $T_i \rightarrow T_j$ if T_i executes read (Q) before T_j executes write (Q).
 3. Create a node $T_i \rightarrow T_j$ if T_i executes write (Q) before T_j executes write (O).

Precedence graph

Precedence graph for Schedule S



- If a precedence graph contains a single edge $T_i \rightarrow T_j$, then all the instructions of T_i are executed before the first instruction of T_j is executed.
- If a precedence graph for schedule S contains a cycle, then S is non-serializable. If the precedence graph has no cycle, then S is known as serializable.

Example

T1

$R(x)$

T2

$R(y)$

$W(y)$

$W(z)$

T3

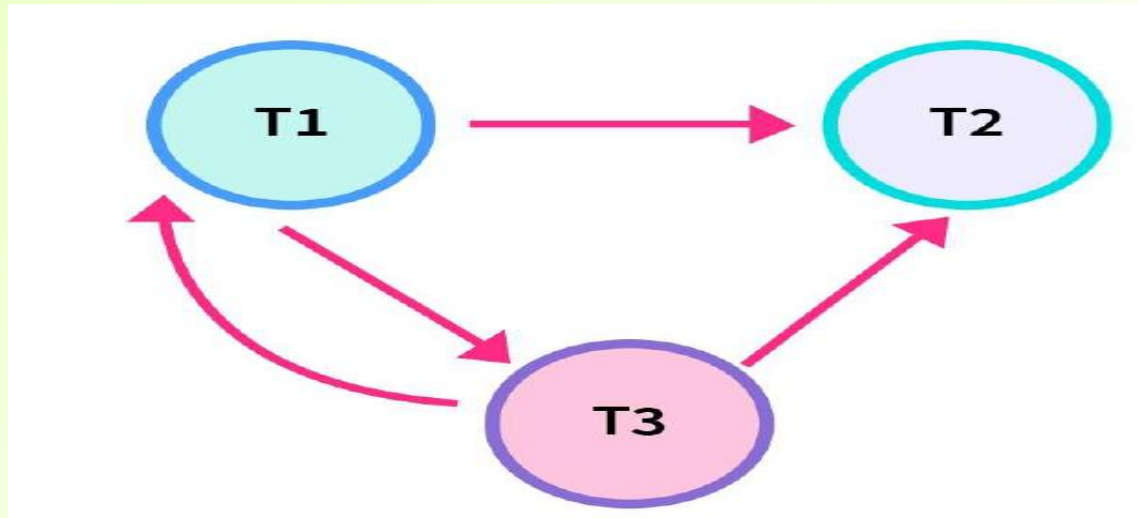
$R(z)$

$W(z)$

$W(x)$

$W(x)$

Example, con't...



- **Non-serializable schedule.** $R(x)$ of T1 conflicts with $W(x)$ of T3, so there is a directed edge from T1 to T3. $R(y)$ of T1 conflicts with $W(y)$ of T2, so there is a directed edge from T1 to T2. $W(x)$ of T3 conflicts with $W(x)$ of T1, so there is a directed edge from T3 to T1. Similarly, we will make edges for every conflicting pair. Now, as the cycle is formed, the transactions **cannot be conflict serializable**.

View Serializability

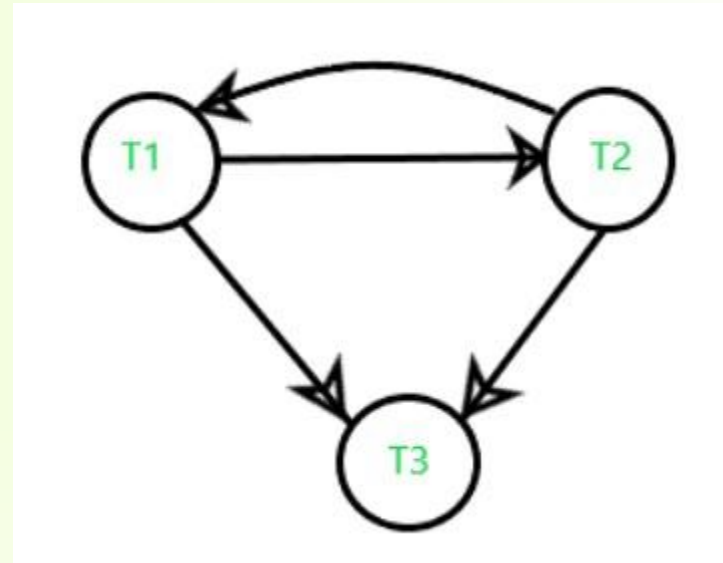
- The **View serializability** is a concept that is used to compute whether schedules are View-Serializable or not.
- A schedule is said to be View-Serializable if it is view equivalent to a Serial Schedule (where no interleaving of transactions is possible).
- There may be some schedules that are not **Conflict-Serializable** but still gives a consistent result.
- In such a case we cannot predict whether a schedule would be consistent or inconsistent.
- For Conflict-Serializability, we can say that a schedule is Not Conflict-Serializable if a schedule's precedence graph contains a cycle/loop but it may results in serializable schedule.
- So, to address such cases we use the concept of **View-Serializability**.

View Serializability

- Example: Schedule S

T_1	T_2	T_3
a=100 read(a)		
	a=a-40 write(a) //60	
a=a-40 write(a) //20		
		a=a-20 write(a) //0

Conflict Precedence Graph



Here, graph contains cycle/loop which means it is **not conflict-serializable** but it does not mean that it cannot be consistent and equivalent to the serial schedule

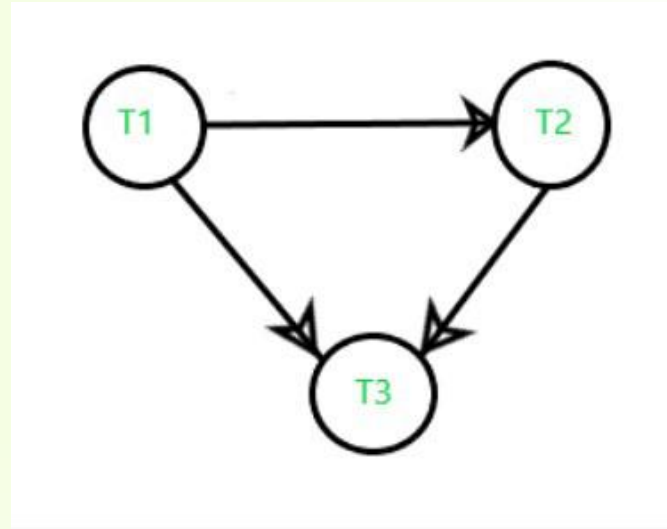
View Serializability

- **Example: Schedule S'**

In the previous example (S) if we do swapping among some transaction's operation so our table will be:

T₁	T₂	T₃
a=100 read(a) //100		
a=a-40 write(a) //60		
	a=a-40 write(a) //20	
		a=a-20 write(a) //0

Its Precedence Graph is



The graph does not contain any cycle/loop, which means it is conflict serializable (equivalent to serial schedule, consistent) and the final result is coming the same as the first table.

i.e. operation a=a-20 results in a= 0 which is same as previous schedule.

View Serializability

- If a schedule is **Conflict-serializable** so we can say that It would be –
 - Equivalent to a serial schedule,
 - Consistent, and
 - also a View-Serializable.
- But what if it is **non-conflict serializable** (precedence graph contains loop). In this situation, we cannot predict whether it is consistent and serializable or not.
- We have to check for the schedule for **View Serializability**. The schedule may or may not be view serializable.

View Serializability

How to ensure a schedule is view serializable or not?

- First of all, check whether the given schedule is Non-Conflict Serializable or Conflict-Serializable.
- If the given schedule is **conflict serializable** then the given schedule must be a **view serializable**.
- If the given schedule is **non-conflict serializable**, then it **may or may not be view serializable**.
 - If **NO BLIND WRITES** exists, then the schedule must be a **non-View-Serializable schedule**.
 - If there exists any **blind write**, then, in that case, the schedule may or may not be view serializable.

Blind write: Performing the Writing operation (updatation), without reading operation on a data is known as a blind write.

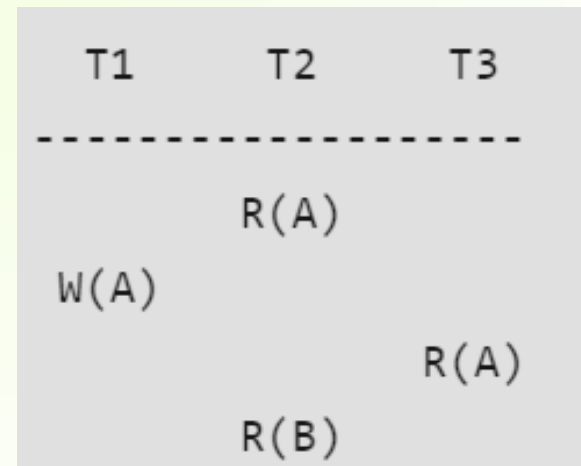
View Serializability

Condition of schedules to View-equivalent

View Serializability: A Schedule is called view serializable if it is view equal to a serial schedule.

Two schedules S1 and S2 are said to be view-equivalent if the below conditions are satisfied:

- 1) Initial Read:** If a transaction T1 reads data item A from the database in S1 then in S2 also T1 should read A from database.



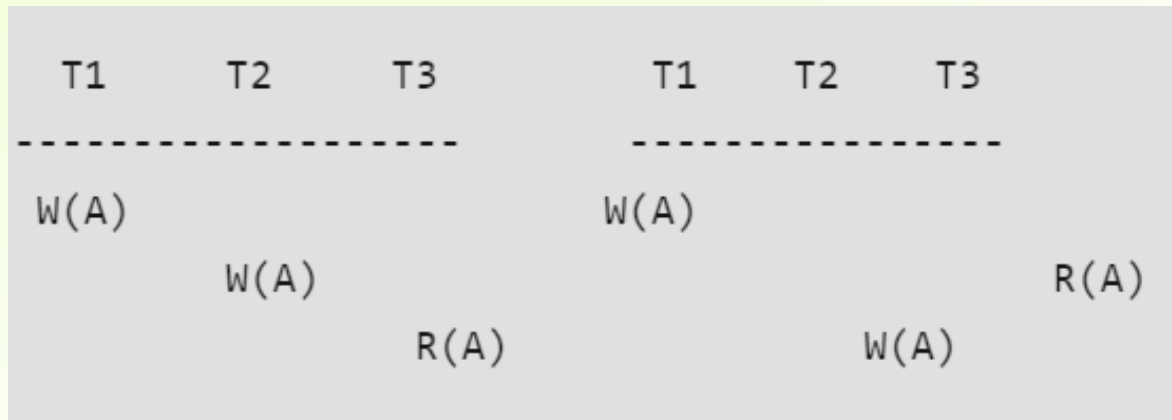
Here, Transaction T2 is reading A from the database.

View Serializability

Condition of schedules to View-equivalent

Two schedules S1 and S2 are said to be view-equivalent if the below conditions are satisfied :

2) Updated Read: If T_i is reading A which is updated by T_j in S1 then in S2 also T_i should read A which is updated by T_j .



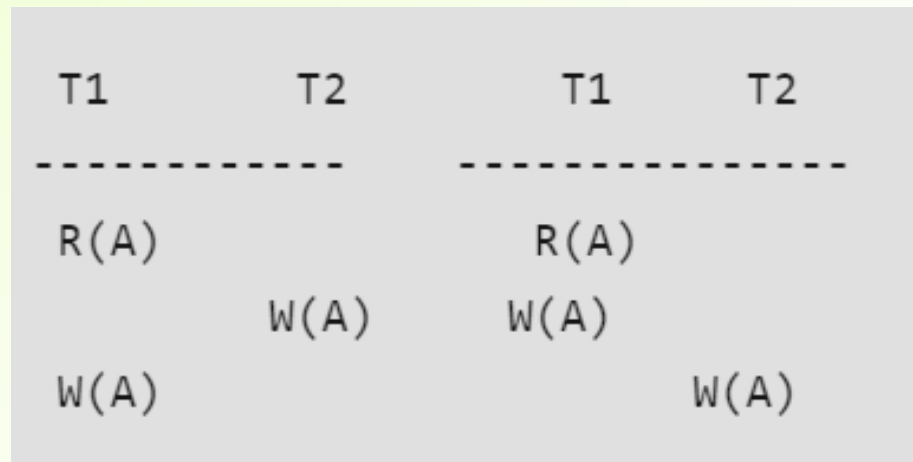
Above two schedules are not view-equivalent as in S1: T3 is reading A updated by T2, in S2 T3 is reading A updated by T1.

View Serializability

Condition of schedules to View-equivalent

Two schedules S1 and S2 are said to be view-equivalent if the below conditions are satisfied :

3) Final Write operation: If a transaction T1 updated A at last in S1, then in S2 also T1 should perform final write operations.



Above two schedules are not view-equivalent as Final write operation in S1 is done by T1 while in S2 done by T2.

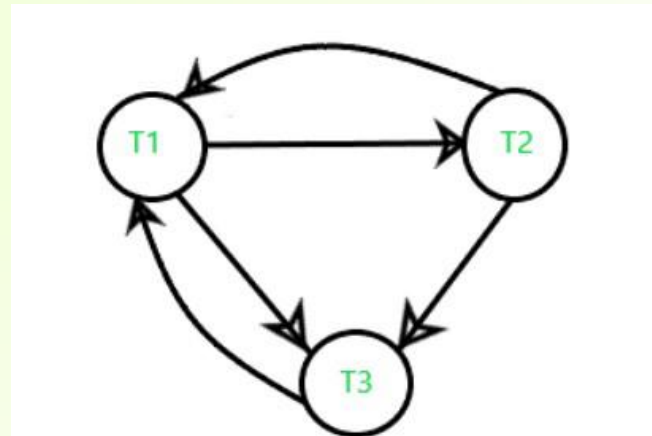
View Serializability

Example:

S: read1(A), write2(A), read3(A), write1(A), write3(A)

T ₁	T ₂	T ₃
read(a)		
	write(a)	
		read(a)
write(a)		
		write(a)

Conflict Precedence Graph



it is conflict serializable

Check whether the Schedule S contains any **blind write**?

We found that the schedule S contains a blind-write write2(a) in transaction T2.

Therefore, **the schedule S may or may not be View-Serializable.**

Now, we will check for view equivalent schedule.

View Serializability

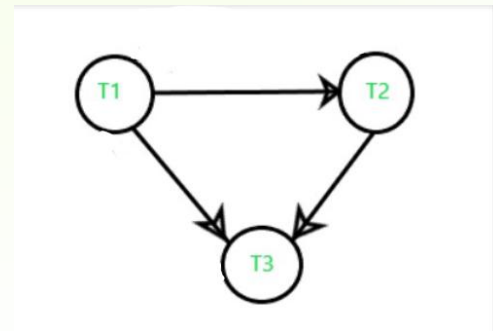
Example:

S: read1(A), write2(A), read3(A), write1(A), write3(A)

T_1	T_2	T_3
read(a)		
	write(a)	
		read(a)
write(a)		
		write(a)

- Transaction T_1 first reads data_item “a” and transaction T_2 first updates(write) “a”.
- So, the transaction T_1 must execute before T_2 .
- In that way, we get the dependency $(T_1 \rightarrow T_2)$ in the graph.
- And, the final update(write) on “a” is made by transaction T_3 .
- So, transaction T_3 must execute after all the other **transactions**(T_1, T_2).
- Thus, we get the dependency $(T_1, T_2) \rightarrow T_3$ in the graph shown below:

As there is no cycle/loop in the dependency graph, the schedule S' is View-Serializable.



Thank You